My HW #4.3 Reproducible Data Analysis

Shreya Chandanshive

2026-04-14

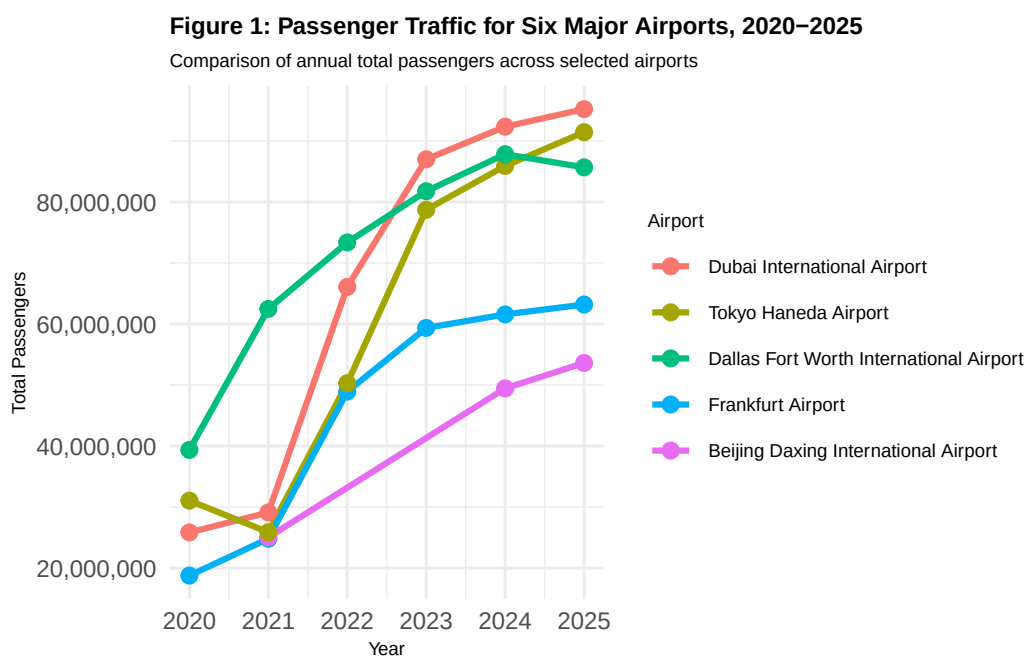
0.1 Busiest Airports Analysis:

Busiest Airports Table:

Table 1: Passenger Traffic for Selected Busiest Airports (2020-2025)

Airport	2020	2021	2022	2023	2024	2025
Dubai International Airport	25836771	29110609	66069981	86994365	92331506	95200000
Tokyo Haneda Airport	31055210	25876429	50290705	78719302	85900167	91430000
Dallas Fort Worth International Airport	39364990	62465756	73362946	81755538	87817864	85660127
Frankfurt Airport	18768601	24812849	48918482	59355389	61564957	63189666
Beijing Daxing International Airport	NA	25051012	NA	NA	49441029	53618949

Busiest Airports Plot/Graph:



Busiest Airports Narrative:

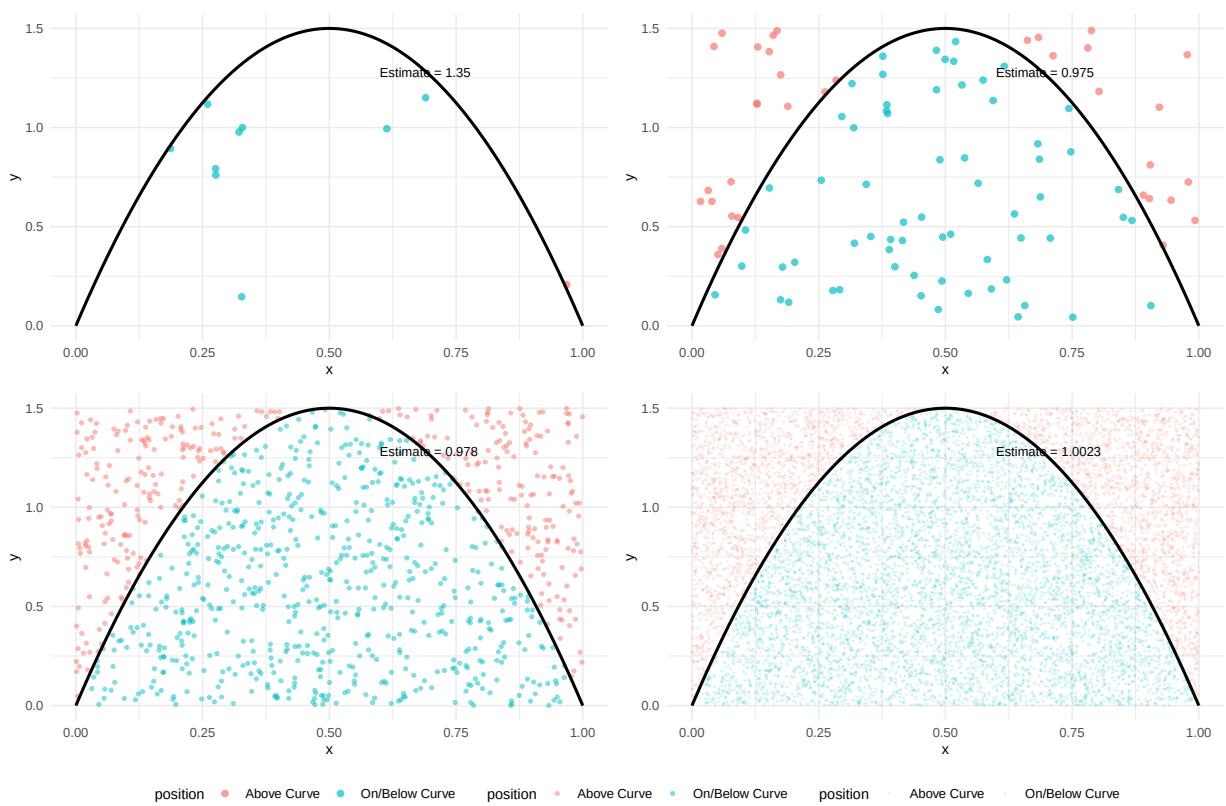
Both Table 1 and Figure 1 illustrate changes in passenger traffic across the selected busiest airports from 2020 to 2025, with the table providing exact values and the line graph emphasizing overall trends. As shown in Table 1, all airports experienced reduced passenger volumes in 2020, followed by consistent increases in subsequent years, indicating a gradual recovery in global air travel. This pattern is clearly reinforced in Figure 1, where most airports display upward-sloping lines after 2021. Dubai International Airport and Tokyo Haneda Airport exhibit some of the most rapid growth, particularly between 2021 and 2023, as seen in the steep increases in their lines in Figure 1. Dallas Fort Worth International Airport shows steady and sustained growth over time, while Frankfurt Airport follows a slower but consistent upward trajectory. Beijing Daxing International Airport, which has limited early data in

Table 1, demonstrates noticeable growth in the later years, suggesting ongoing development and expansion. Overall, the narrative aligns with both the visual and tabular representations by highlighting key trends, including the initial decline in 2020, varying recovery rates across airports, and the continued growth in passenger traffic through 2025.

0.2 Monte Carlo Numerical Integration:

Monte Carlo Plot/Graph:

Figure 1: Monte Carlo integration results for increasing sample sizes.



Monte Carlo Narrative Text:

Figure 1 shows four scatter plots demonstrating Monte Carlo integration with increasing sample sizes (n = 10, 100, 1000, and 10000). In each plot, points are randomly distributed within a rectangle (x from 0 to 1, y from 0 to 1.5), with the black curve representing the Beta(2,2) density. Points are colored teal if they fall on or below the curve and red if they fall above it. The proportion of points below the curve is used to estimate the integral, which is displayed in each panel.

As the sample size increases, the points more clearly approximate the shape of the curve and the estimate becomes more stable and accurate. The smaller samples show greater variability, while the larger samples produce estimates that closely converge to the true value, illustrating the effectiveness of Monte Carlo integration.

0.3 Planning and Prompting GenAI:

Plan: Tidying and Visualizing the Calcium Study Data

Goals The main goals of this report are to:

1. Import and inspect the calcium dataset,
2. Tidy and wrangle the data into a usable structure,
3. Compute summary statistics by group and year, and
4. Create a polished visualization that supports interpretation of the study context.

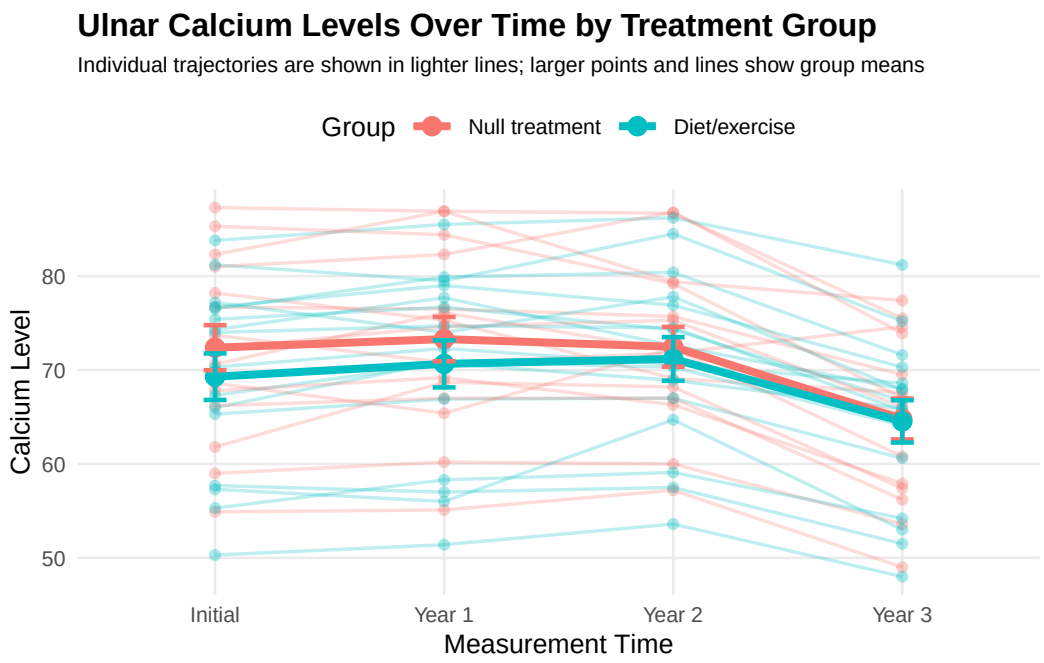
Needs

This analysis uses: 1. Tidyverse for wrangling and plotting 2. A CSV file containing the calcium measurements 3. A tidy structure with one row per subject per year 4. A visualization that highlights group trends over time.

Instruction/Steps:

1. Load packages
2. Read the data
3. Tidy and Wrangle the Data
4. Summarize the Data
5. Create a Polished Visualization

My visualization



My narrative: This plot shows how calcium levels change over four years for both the null treatment and diet/exercise groups while also displaying individual variation. Both groups start at similar levels, with the null group slightly higher, and show small fluctuations through Years 1 and 2, where their averages become nearly the same. By Year 3, both groups experience a noticeable decline. Overall, calcium levels decrease over time in both groups, but the diet/exercise group appears to maintain slightly more stable levels earlier on, although the long-term difference between groups is modest.

Informed AI response: The visualization indicates that both groups begin with comparable calcium levels, though the null treatment group is slightly higher at baseline. Over the first two years, the diet/exercise group shows a slight increase or stabilization, closing the gap between groups. By Year 2, the averages are nearly identical. However, both groups experience a clear decline by Year 3. Although there is substantial individual variability, the diet/exercise group appears to maintain calcium levels more effectively in the earlier years, suggesting a modest short-term benefit of the intervention, even though long-term differences between groups remain limited.

Generic AI response: The plot shows how calcium levels change over time for two groups. Both groups follow a similar pattern, with small changes in the early years and a noticeable decrease by the final year. While there is some variation between individuals, the overall trends for the two groups appear fairly similar.

Here is the link to ChatGPT's response in a word doc:

https://pennstateoffice365-my.sharepoint.com/:w:/r/personal/svc6521_psu_edu/Documents/Document2.docx?d=w480d3d17ce0643a5813a169417a9bfe4&csf=1&web=1&e=dhpwLP

Compare/Contrast

The two visualizations present the same underlying dataset but differ in how they emphasize interpretation and clarity. My plot is more polished and structured, using cleaner labeling, a clearer legend placement, and a balanced visual hierarchy. It separates individual trajectories (lighter lines)

from group means (bold lines and points), making it easier to distinguish between variability and overall trends. In contrast, ChatGPT's plot appears more cluttered due to overlapping colors and less refined styling, which makes it slightly harder to immediately interpret the key patterns.

Beyond visual differences, the plots reflect different approaches to representing the "case" versus the "situation." My plot better communicates the case-level structure and it clearly distinguishes treatment groups and highlights their comparative trends over time. ChatGPT's plot leans more toward showing the raw situation, emphasizing all individual trajectories equally, which preserves detail but reduces interpretability. This contrast illustrates an important tradeoff in data visualization: balancing completeness with clarity.

From working with genAI/LLM tools in this activity I learned how initial outputs generated by AI may be technically correct but not optimally structured for communication. Refinement are always necessary and AI cannot always be relied on for things such as adjusting aesthetics, clarifying labels, and explicitly separating summary from raw data—is necessary to transform a plot into something analytically meaningful. Any small technical issues can significantly impact readability, which reinforces the importance of understanding the underlying code rather than relying solely on generated outputs. Overall, the activity highlights that while genAI tools accelerate the process, effective results still depend on human judgment in refining and interpreting the visualization.

0.4 Self Reflection:

Throughout this semester, I've learned a lot from this course- the main thing being how to code in R. I've never worked with R before, but it was a little similar to python which made it easier and I got to work with different syntax. From the homework assignments, I've learned how important it is to make a detailed plan in order to execute my goals. Because of the pre-planning, it made it easier to know exactly what I was doing. I will definitely be using this tactic in the future for other projects and assignments in my life. With this, I also learned the basics of R studio of how to install and load packages, use the console, start a project, and import files from my desktop as well as through the internet links. From homework 3.4, I learned how to create tables, plots, alt texts and long descriptions, as well as creating narratives from what I learned from the created visuals. Aside from assignments, I got to experience making and talking to friends in the class because I usually don't do that.

GenAI Appendix I used ChatGPT as my source of GenAI and it was used in question 4. The version I used was ChatGPT plus. It was used on April 14th, 2026. The prompt that was given is in the word document that I linked in the GenAI planning section. The response is included in this as well.

1 Code Appendix

```
# Busiest Airports Data Wrangling ----

rm(list = ls())

# Step 1: Load Packages ----
library(tidyverse)
library(rvest)
library(janitor)
library(readr)

# Step 2: Read the Wikipedia page ----
wiki_url <- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"

airport_tables <- wiki_url |>
  read_html() |>
  html_elements("table.wikitable") |>
  html_table(fill = TRUE)

# Step 3: Keep the yearly tables for 2025-2020 ----
years <- 2025:2020

yearly_airport_data <- map2(
  airport_tables[1:6],
  years,
  \(tbl, yr) {
    tbl |>
      clean_names() |>
      mutate(year = yr)
  }
) |>
  list_rbind()

# Step 4: Keep only needed columns ----
airports_tidy <- yearly_airport_data |>
  select(year, airport, code_iata_icao, totalpassengers)

# Step 5: Then clean the text fields ----
airports_tidy <- airports_tidy |>
```

```r
  mutate(
    airport = str_squish(str_remove_all(airport, "\\[.*?\\]")),
    code_iata_icao = str_squish(str_remove_all(code_iata_icao, "\\[.*?\\]")),
    totalpassengers = parse_number(totalpassengers)
  )

# Step 6: Separate the airport codes ----
airports_tidy <- airports_tidy |>
  separate_wider_delim(
    code_iata_icao,
    delim = "/",
    names = c("iata", "icao"),
    too_few = "align_start"
  )

# Step 7: Then only keep the six airports ----
selected_airports <- c(
  "Hartsfield-Jackson Atlanta International Airport",
  "Frankfurt Airport",
  "Beijing Daxing International Airport",
  "Dubai International Airport",
  "Tokyo Haneda Airport",
  "Dallas Fort Worth International Airport"
)

airports_tidy <- airports_tidy |>
  filter(airport %in% selected_airports)

# Step 8: Then rename and arrange the final tidy data ----
airports_tidy <- airports_tidy |>
  rename(total_passengers = totalpassengers) |>
  mutate(airport = factor(airport, levels = selected_airports)) |>
  arrange(airport, year) |>
  select(year, airport, iata, icao, total_passengers)

# Step 9: Finally view the final data ----
airports_tidy
# Busiest Airports Table Code

# Step 1: Load Package ----
library(knitr)
library(dplyr)
library(tidyr)
library(stringr)

# Step 2: Clean and Prepare Data for Table ----
airports_table <- airports_tidy |>
```

```r
  # Remove commas and citations from passenger values, convert to numeric
  mutate(
    total_passengers = str_remove_all(total_passengers, "\\[.*?\\]"),
    total_passengers = str_remove_all(total_passengers, ","),
    total_passengers = as.numeric(total_passengers)
  ) |>

  # Pivot to wide format and make the years as columns
  pivot_wider(
    names_from = year,
    values_from = total_passengers
  ) |>

  # Sort by most recent year (2025)
  arrange(desc(`2025`)) |>

  # Keep only relevant columns
  select(airport, `2020`, `2021`, `2022`, `2023`, `2024`, `2025`)

# Step 3: Create Table ----
kable(
  airports_table,
  col.names = c("Airport", "2020", "2021", "2022", "2023", "2024", "2025"),
  caption = "Passenger Traffic for Selected Busiest Airports (2020-2025)"
)


# Busiest Airports Plot ----

# Step 1: Load Packages ----
library(ggplot2)
library(dplyr)

# Step 2: Prepare the data for plotting ----
airports_plot_data <- airports_tidy |>
  mutate(
    airport = as.character(airport),
    airport = factor(
      airport,
      levels = c(
        "Hartsfield-Jackson Atlanta International Airport",
        "Dubai International Airport",
        "Tokyo Haneda Airport",
        "Dallas Fort Worth International Airport",
        "Frankfurt Airport",
        "Beijing Daxing International Airport"
      )
```

```r
    )
  )

# Step 3: Create the plot/graph ----
ggplot(airports_plot_data, aes(x = year, y = total_passengers, color = airport)) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 2.5) +
  scale_x_continuous(breaks = 2020:2025) +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "Figure 1: Passenger Traffic for Six Major Airports, 2020-2025",
    subtitle = "Comparison of annual total passengers across selected airports",
    x = "Year",
    y = "Total Passengers",
    color = "Airport"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 9),
    plot.subtitle = element_text(size = 7),
    axis.title = element_text(size = 7),
    legend.title = element_text(size = 7),
    legend.text = element_text(size = 7),
    legend.position = "right"
  )
# Monte Carlo Numerical Integration Small Multiple ----

# Step 1: Load Packages ----
library(tidyverse)
library(patchwork)

# Step 2: Monte Carlo Simulation Function ----
monte_carlo_sim <- function(n, x_min, x_max, y_min, y_max) {
  x_vals <- runif(n, min = x_min, max = x_max)
  y_vals <- runif(n, min = y_min, max = y_max)

  data.frame(
    x = x_vals,
    y = y_vals
  )
}

# Step 3: Create a reusable wrangling + plotting function ----
make_mc_plot <- function(n) {

  # Generate points
  mc_points <- monte_carlo_sim(
```

```r
  n = n,
  x_min = 0,
  x_max = 1,
  y_min = 0,
  y_max = 1.5
)

# Classify points
mc_points <- mc_points |>
  mutate(
    curve_y = dbeta(x, shape1 = 2, shape2 = 2),
    position = if_else(y <= curve_y, "On/Below Curve", "Above Curve")
  )

# Calculate estimate
rect_area <- 1 * 1.5

mc_estimate <- mc_points |>
  summarize(
    proportion_under = mean(position == "On/Below Curve"),
    estimated_integral = proportion_under * rect_area
  )

estimate_label <- paste0(
  "Estimate = ",
  round(mc_estimate$estimated_integral, 4)
)

# Adjust point size and transparency based on n
point_size <- case_when(
  n <= 100 ~ 1.8,
  n <= 1000 ~ 1.0,
  TRUE ~ 0.3
)

point_alpha <- case_when(
  n <= 100 ~ 0.7,
  n <= 1000 ~ 0.5,
  TRUE ~ 0.15
)

# Make plot
ggplot(mc_points, aes(x = x, y = y, color = position)) +
  geom_point(alpha = point_alpha, size = point_size) +
  stat_function(
    fun = dbeta,
    args = list(shape1 = 2, shape2 = 2),
```

```r
      xlim = c(0, 1),
      linewidth = 1,
      color = "black"
    ) +
    annotate(
      "text",
      x = 0.60,
      y = 1.28,
      label = estimate_label,
      hjust = 0,
      size = 3.2
    ) +
    coord_cartesian(xlim = c(0, 1), ylim = c(0, 1.5)) +
    theme_minimal() +
    theme(
      plot.title = element_text(face = "bold", size = 11),
      axis.title = element_text(size = 10),
      axis.text = element_text(size = 9),
      legend.position = "bottom",
      legend.title = element_text(size = 10),
      legend.text = element_text(size = 9)
    )
}

# Step 4: Save each plot as its own object ----
plot10 <- make_mc_plot(10)
plot100 <- make_mc_plot(100)
plot1000 <- make_mc_plot(1000)
plot10000 <- make_mc_plot(10000)

# Step 5: Combine plots into a clean 2x2 small multiple ----
combined_plot <- (plot10 + plot100) / (plot1000 + plot10000) +
  plot_layout(guides = "collect") &
  theme(legend.position = "bottom")

combined_plot
# Step 1: Load packages ----
library(tidyverse)

# Step 2: Load data ----
calcium_raw <- read_csv("calcium.csv")

# Step 3: Split groups safely ----
null_df <- calcium_raw |>
  select(1:4) |>
  setNames(c("Initial", "Year 1", "Year 2", "Year 3")) |>
  mutate(
```

```r
    subject_id = row_number(),
    group = "Null treatment"
  )

diet_df <- calcium_raw |>
  select(5:8) |>
  setNames(c("Initial", "Year 1", "Year 2", "Year 3")) |>
  mutate(
    subject_id = row_number(),
    group = "Diet/exercise"
  )

# Step 4: Combine and tidy ----
calcium_tidy <- bind_rows(null_df, diet_df) |>
  pivot_longer(
    cols = c("Initial", "Year 1", "Year 2", "Year 3"),
    names_to = "year",
    values_to = "calcium"
  ) |>
  mutate(
    year = factor(year, levels = c("Initial", "Year 1", "Year 2", "Year 3")),
    group = factor(group, levels = c("Null treatment", "Diet/exercise"))
  )

# Step 5: Summary data ----
calcium_summary <- calcium_tidy |>
  group_by(group, year) |>
  summarize(
    mean_calcium = mean(calcium, na.rm = TRUE),
    se = sd(calcium, na.rm = TRUE) / sqrt(n()),
    .groups = "drop"
  )

# Step 6: Plot ----
ggplot(
  calcium_tidy,
  aes(
    x = year,
    y = calcium,
    group = interaction(group, subject_id),
    color = group
  )
) +
  geom_line(alpha = 0.25, linewidth = 0.6) +
  geom_point(alpha = 0.35, size = 1.5) +
  geom_line(
    data = calcium_summary,
```

```r
    aes(x = year, y = mean_calcium, group = group, color = group),
    linewidth = 1.4,
    inherit.aes = FALSE
  ) +
  geom_point(
    data = calcium_summary,
    aes(x = year, y = mean_calcium, color = group),
    size = 3,
    inherit.aes = FALSE
  ) +
  geom_errorbar(
    data = calcium_summary,
    aes(
      x = year,
      y = mean_calcium,
      ymin = mean_calcium - se,
      ymax = mean_calcium + se,
      color = group
    ),
    width = 0.1,
    linewidth = 0.8,
    inherit.aes = FALSE
  ) +
  labs(
    title = "Ulnar Calcium Levels Over Time by Treatment Group",
    subtitle = "Individual trajectories are shown in lighter lines; larger points and lines sho
    x = "Measurement Time",
    y = "Calcium Level",
    color = "Group"
  ) +
  theme_minimal(base_size = 10) +
  theme(
    plot.title = element_text(face = "bold", size = 12),
    plot.subtitle = element_text(size = 8),
    legend.position = "top",
    panel.grid.minor = element_blank()
  )
```